

C134: Computer Organisation and Architecture

Unit 1-fundamentals of digital logic and logic circuits

Dr. Anjaneyulu Pasala

Dr. Mohan Kumar S

Department of CSE(AI&ML)

MSRIT – Bangalore

Information in digital computers

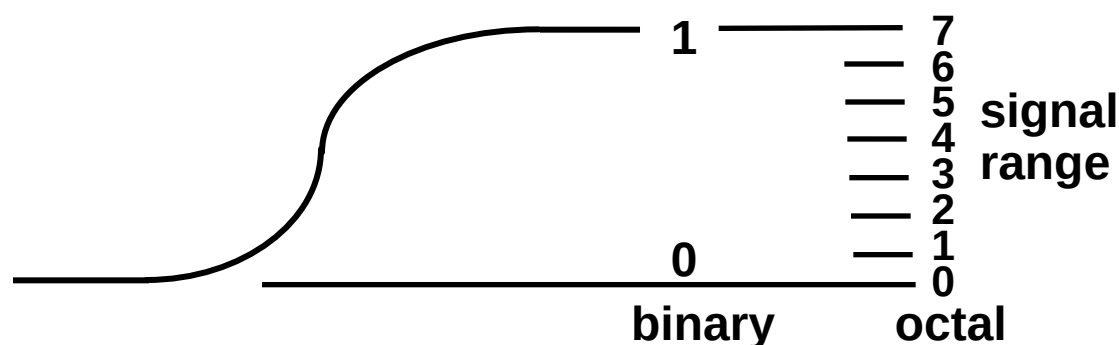
- Information in digital computers is stored and processed by electronic networks called logic gates.
- Logic circuits operate on binary values called 0 and 1

LOGIC GATES

Digital Computers

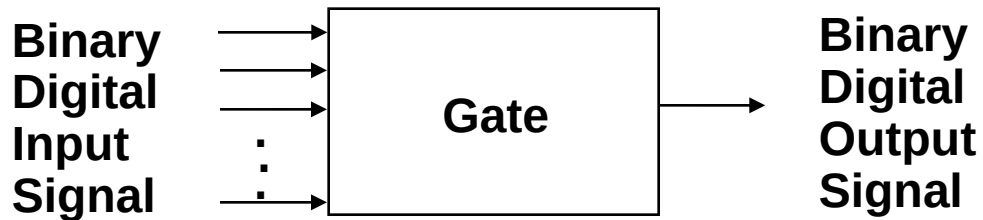
- Imply that the computer deals with digital information, i.e., it deals with the information that is represented by binary digits
- Why *BINARY* ? instead of Decimal or other number system ?

* Consider electronic signal



A logic gate is a device that performs a Boolean function which is a logical operation performed on one or more binary inputs that produces a single binary output.

BASIC LOGIC GATE



The logic gates are built using diodes or transistors acting as electronic switches.

Today, MOSFETs (Metal-Oxide Semiconductor Field-Effect Transistors)

Logic Circuits

- A set of logic gates are cascaded to form a logic circuit such as multiplexers, registers, arithmetic logic units (ALUs) and computer memory. Of course microprocessor.
- Cascade of logic gates used in the construction of physical model of all Boolean logic and therefore, all the algorithms and mathematics.

Types of Basic Logic Blocks

- **Combinational Logic Block**

Logic Blocks whose output logic value depends only on the input logic values

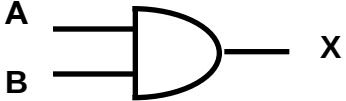
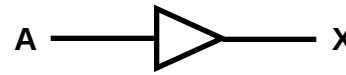
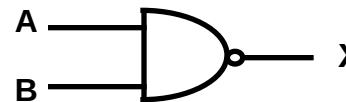
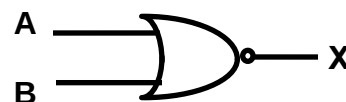
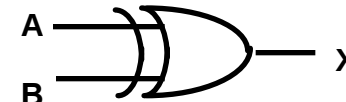
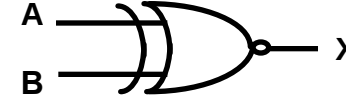
- **Sequential Logic Block**

Logic Blocks whose output logic value depends on the input values and the state (stored information) of the blocks

Functions of Gates can be described by

- **Truth Table**
- **Boolean Function**
- **Karnaugh Map**

COMBINATIONAL GATES

Name	Symbol	Function	Truth Table															
AND		$X = A \cdot B$ or $X = AB$	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	X	0	0	0	0	1	0	1	0	0	1	1	1
A	B	X																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$X = A + B$	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	X	0	0	0	0	1	1	1	0	1	1	1	1
A	B	X																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
I		$X = A'$	<table><tr><th>A</th><th>X</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	X	0	1	1	0									
A	X																	
0	1																	
1	0																	
Buffer		$X = A$	<table><tr><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	A	X	0	0	1	1									
A	X																	
0	0																	
1	1																	
NAND		$X = (AB)'$	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	X	0	0	1	0	1	1	1	0	1	1	1	0
A	B	X																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$X = (A + B)'$	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	X	0	0	1	0	1	0	1	0	0	1	1	0
A	B	X																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
XOR Exclusive OR		$X = A \oplus B$ or $X = A'B + AB'$	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	X	0	0	0	0	1	1	1	0	1	1	1	0
A	B	X																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
XNOR Exclusive NOR or Equivalence		$X = (A \oplus B)'$ or $X = A'B' + AB$	<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	X	0	0	1	0	1	0	1	0	0	1	1	1
A	B	X																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

BOOLEAN ALGEBRA

Boolean Algebra

- * Algebra with Binary(Boolean) Variable and Logic Operations
- * Boolean Algebra is useful in Analysis and Synthesis of Digital Logic Circuits
 - Input and Output signals can be represented by Boolean Variables, and
 - Function of the Digital Logic Circuits can be represented by Logic Operations, i.e., Boolean Function(s)
 - From a Boolean function, a logic diagram can be constructed using AND, OR, and I

Truth Table

- * The most elementary specification of the function of a Digital Logic Circuit is the Truth Table
 - Table that describes the Output Values for all the combinations of the Input Values, called *MINTERMS*
 - n input variables $\rightarrow 2^n$ minterms

LOGIC CIRCUIT DESIGN

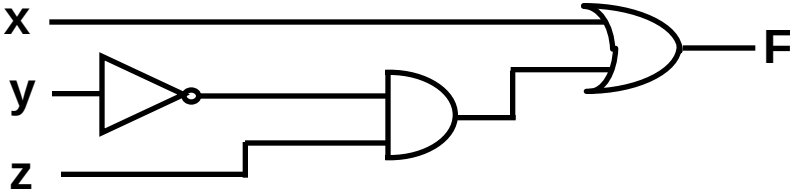
Truth Table

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Boolean Function

$F = x + y'z$

Logic Diagram



BASIC IDENTITIES OF BOOLEAN ALGEBRA

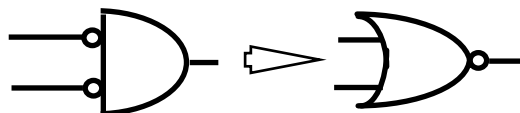
[1] $x + 0 = x$	[2] $x \cdot 0 = 0$
[3] $x + 1 = 1$	[4] $x \cdot 1 = x$
[5] $x + x = x$	[6] $x \cdot x = x$
[7] $x + x' = 1$	[8] $x \cdot x' = 0$
[9] $x + y = y + x$	[10] $xy = yx$
[11] $x + (y + z) = (x + y) + z$	[12] $x(yz) = (xy)z$
[13] $x(y + z) = xy + xz$	[14] $x + yz = (x + y)(x + z)$
[15] $(x + y)' = x'y'$	[16] $(xy)' = x' + y'$
[17] $(x')' = x$	

[15] and [16] : De Morgan's Theorem

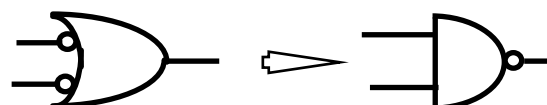
Usefulness of this Table

- Simplification of the Boolean function
 - Derivation of equivalent Boolean functions to obtain logic diagrams utilizing different logic gates
 - Ordinarily ANDs, ORs, and Inverters
 - But a certain different form of Boolean function may be convenient to obtain circuits with NANDs or NORs
- Applications of De Morgans Theorem

$x'y' = (x + y)'$
I, AND → NOR



$x' + y' = (xy)'$
I, OR → NAND



EQUIVALENT CIRCUITS

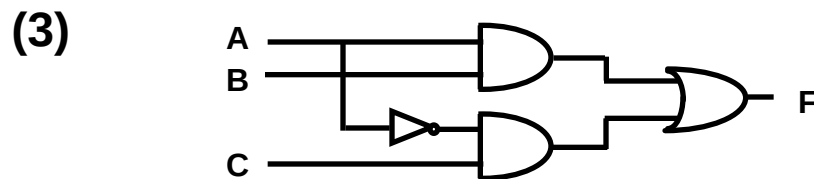
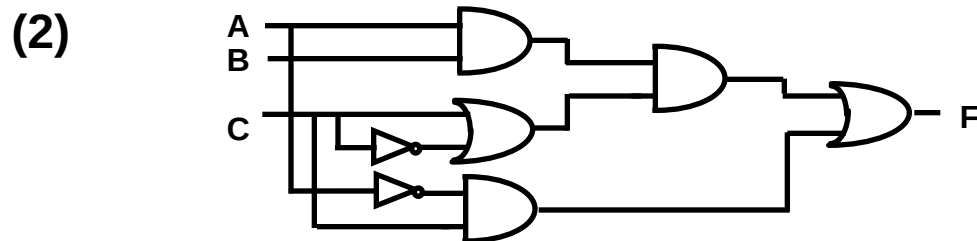
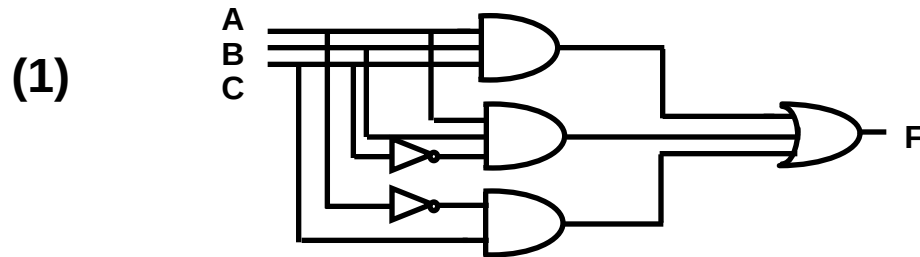
Many different logic diagrams are possible for a given Function

$$F = ABC + ABC' + A'C \quad \dots\dots\dots (1)$$

$$= AB(C + C') + A'C \quad [13] \dots\dots (2)$$

$$= AB \cdot 1 + A'C \quad [7]$$

$$= AB + A'C \quad [4] \dots\dots (3)$$



COMPLEMENT OF FUNCTIONS

A Boolean function of a digital logic circuit is represented by only using logical variables and AND, OR, and Invert operators.

→ Complement of a Boolean function

- Replace all the variables and subexpressions in the parentheses appearing in the function expression with their respective complements

$$A, B, \dots, Z, a, b, \dots, z \Rightarrow A', B', \dots, Z', a', b', \dots, z'$$
$$(p + q) \Rightarrow (p + q)'$$

- Replace all the operators with their respective complementary operators

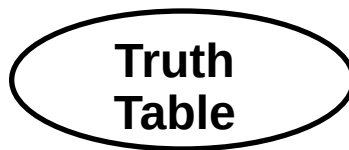
$$\text{AND} \Rightarrow \text{OR}$$
$$\text{OR} \Rightarrow \text{AND}$$

- Basically, extensive applications of the De Morgan's theorem

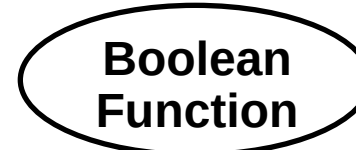
$$(x_1 + x_2 + \dots + x_n)' \Rightarrow x_1' x_2' \dots x_n'$$

$$(x_1 x_2 \dots x_n)' \Rightarrow x_1' + x_2' + \dots + x_n'$$

SIMPLIFICATION



Unique

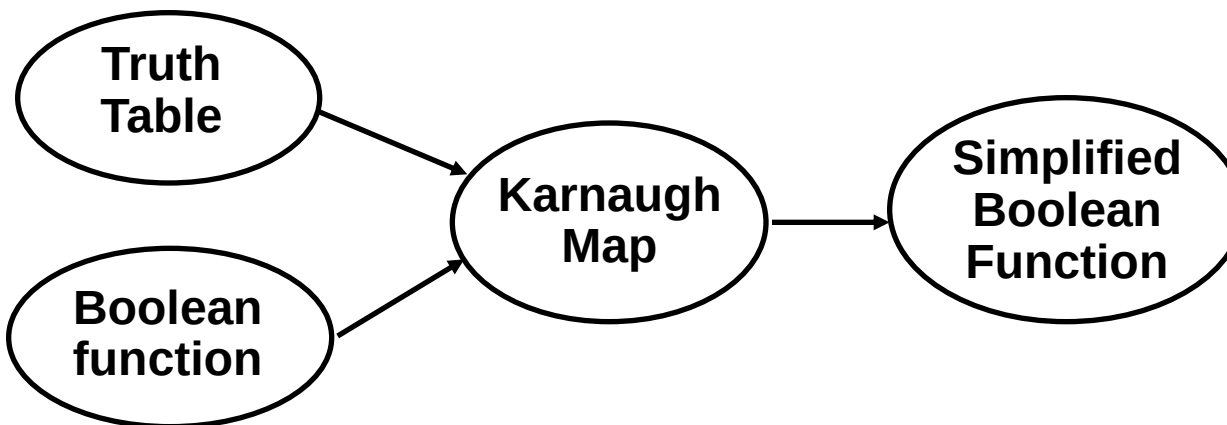


Many different expressions exist

Simplification from Boolean function

- Finding an equivalent expression that is least expensive to implement
- For a simple function, it is possible to obtain a simple expression for low cost implementation
- But, with complex functions, it is a very difficult task

Karnaugh Map (K-map) is a simple procedure for simplifying Boolean expressions.



KARNAUGH MAP

Karnaugh Map for an n-input digital logic circuit (n-variable sum-of-products form of Boolean Function, or Truth Table) is

- Rectangle divided into 2^n cells
- Each cell is associated with a *Minterm*
- An output(function) value for each input value associated with a minterm is written in the cell representing the minterm
→ 1-cell, 0-cell

Each Minterm is identified by a decimal number whose binary representation is identical to the binary interpretation of the input values of the minterm.

Karnaugh Map

x	F
0	1
1	0

x	0	1
0	0	1
1	1	1

Identification of the cell

x	0	1
0	0	1
1	1	1

$F(x) = \sum (1)$
1-cell

value of F

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

x \ y	0	1
0	0	1
1	2	3

x \ y	0	1
0	0	1
1	1	0

$F(x,y) = \sum (1,2)$

KARNAUGH MAP

Karnaugh Map for an n-input digital logic circuit (n-variable sum-of-products form of Boolean Function, or Truth Table) is

- Rectangle divided into 2^n cells
- Each cell is associated with a *Minterm*
- An output(function) value for each input value associated with a minterm is written in the cell representing the minterm
→ 1-cell, 0-cell

Each Minterm is identified by a decimal number whose binary representation is identical to the binary interpretation of the input values of the minterm.

Karnaugh Map

x	F
0	1
1	0

x	
0	0
1	1

Identification of the cell

x	
0	0
1	1

value of F

$F(x) = \sum (1)$

1-cell

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

x \ y	0	1
0	0	1
1	2	3

x \ y	0	1
0	0	1
1	1	0

$F(x,y) = \sum (1,2)$

KARNAUGH MAP

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

		y			
		00	01	11	10
x \ yz	0	0	1	3	2
	1	4	5	7	6

		y			
		00	01	11	10
x \ yz	0	0	1	0	1
	1	1	0	0	0

$F(x,y,z) = \sum (1,2,4)$

u	v	w	x	F
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

		w			
		00	01	11	10
u \ vx	00	0	1	3	2
	01	4	5	7	6
	11	12	13	15	14
	10	8	9	11	10

		w			
		00	01	11	10
u \ vx	00	0	1	1	0
	01	0	0	0	1
	11	0	0	0	1
	10	1	1	1	0

$F(u,v,w,x) = \sum (1,3,6,8,9,11,14)$

MAP SIMPLIFICATION - 2 ADJACENT CELLS -

Rule: $xy' + xy = x(y + y') = x$

Adjacent cells

- binary identifications are different in one bit
- minterms associated with the adjacent cells have one variable complemented each other

Cells (1,0) and (1,1) are adjacent

Minterms for (1,0) and (1,1) are

$$x \cdot y' \rightarrow x=1, y=0$$

$$x \cdot y \rightarrow x=1, y=1$$

$F = xy' + xy$ can be reduced to $F = x$

From the map

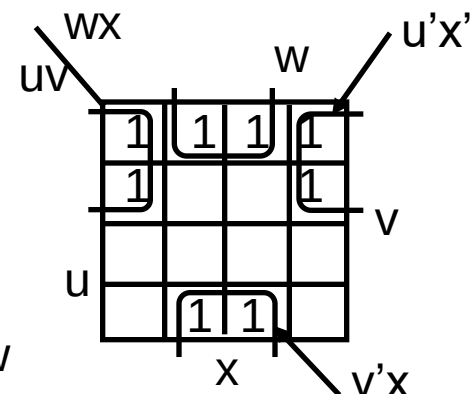
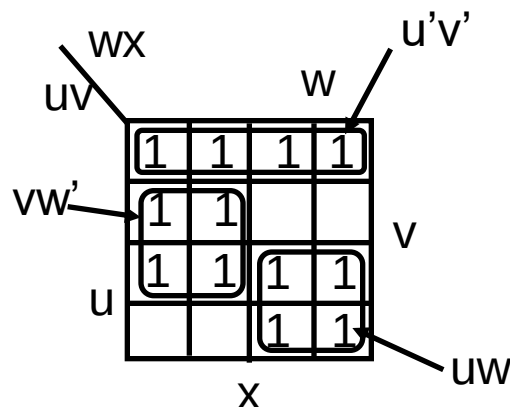
		y	
		0	1
x	0	0	0
	1	1	1

2 adjacent cells xy' and xy
→ merge them to a larger cell x

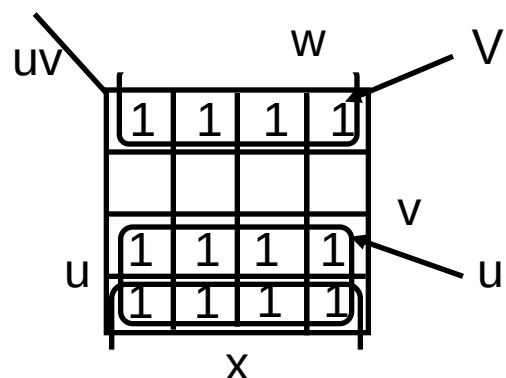
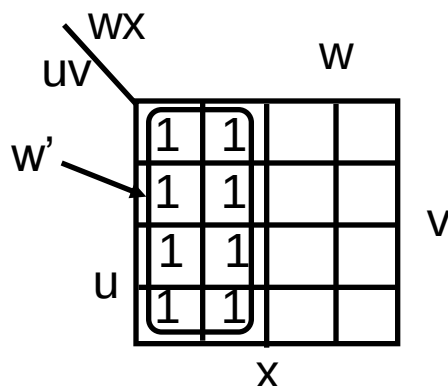
$$\begin{aligned}
 F(x,y) &= \sum (2,3) \\
 &= xy' + xy \\
 &= x
 \end{aligned}$$

MAP SIMPLIFICATION - MORE THAN 2 CELLS -

$$\begin{aligned}
 &u'v'w'x' + u'v'w'x + u'v'wx + u'v'wx' \\
 &= u'v'w'(x' + x) + u'v'w(x + x') \\
 &= u'v'w' + u'v'w \\
 &= u'v'(w' + w) \\
 &= u'v'
 \end{aligned}$$



$$\begin{aligned}
 &u'v'w'x' + u'v'w'x + u'vw'x' + u'vw'x + uvw'x' + uvw'x + uv'w'x' + uv'w'x \\
 &= u'v'w'(x' + x) + u'vw'(x' + x) + uvw'(x' + x) + uv'w'(x' + x) \\
 &= u'(v' + v)w' + u(v' + v)w' \\
 &= (u' + u)w' = w'
 \end{aligned}$$



MAP SIMPLIFICATION

WX uV \	00	01	11	10
00	1	1	0	1
01	0	0	0	0
11	0	1	1	0
10	0	1	0	0



		W			
u	v	1	1	0	1
		0	0	0	0
	x	0	1	1	0
		0	1	0	0

$$F(u,v,w,x) = \sum (0,1,2,9,13,15)$$



(0,1), (0,2), (0,4), (0,8)
Adjacent Cells of 1
 Adjacent Cells of 0
 (1,0), (1,3), (1,5), (1,9)
 ...
 ...
Adjacent Cells of 15
 (15,7), (15,11), (15,13), (15,14)

Merge (0,1) and (0,2)
 --> $u'v'w' + u'v'x'$
Merge (1,9)
 --> $v'w'x$
Merge (9,13)
 --> $uw'x$
Merge (13,15)
 --> uvx

$$F = u'v'w' + u'v'x' + v'w'x + uw'x + uvx$$

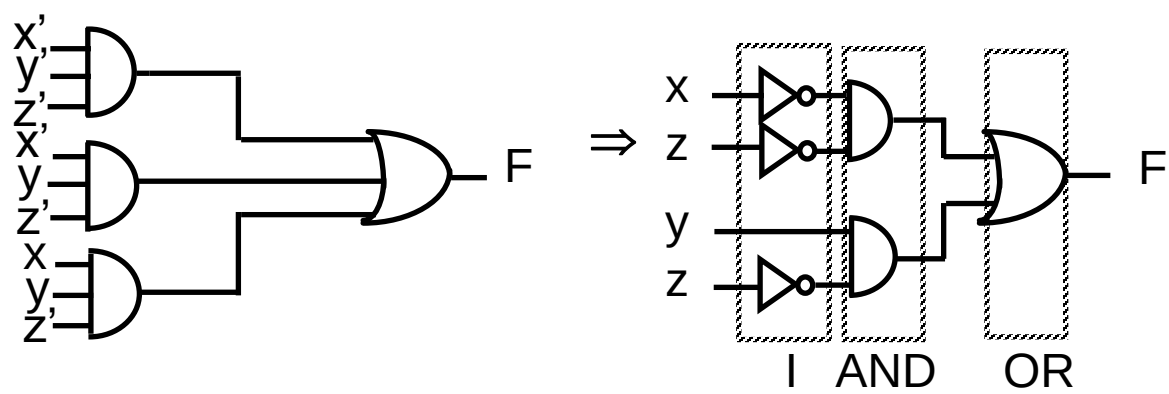
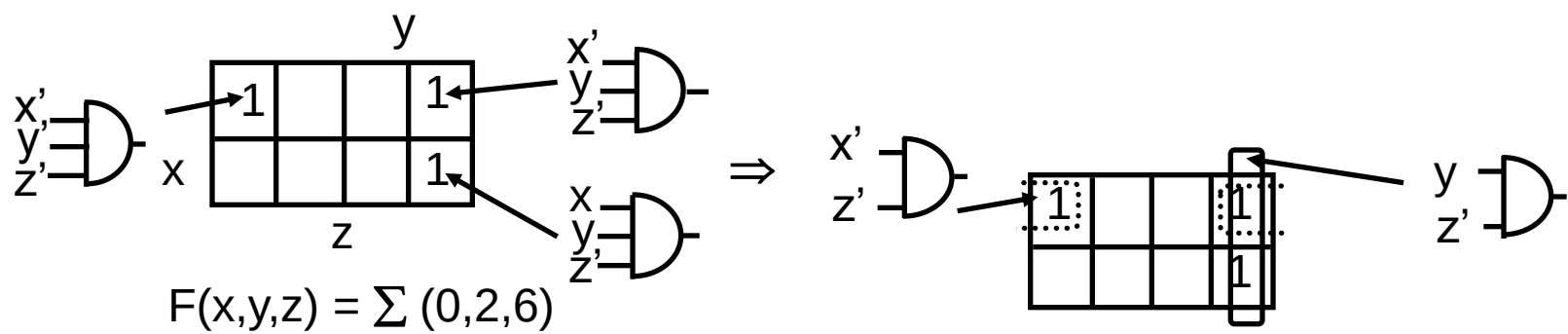
But (9,13) is covered by (1,9) and (13,15)

$$F = u'v'w' + u'v'x' + v'w'x + uvx$$

IMPLEMENTATION OF K-MAPS - Sum-of-Products Form -

Logic function represented by a Karnaugh map can be implemented in the form of I-AND-OR

A cell or a collection of the adjacent 1-cells can be realized by an AND gate, with some inversion of the input variables.

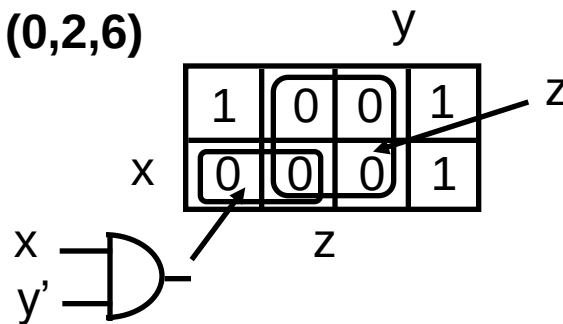


IMPLEMENTATION OF K-MAPS - Product-of-Sums Form -

Logic function represented by a Karnaugh map can be implemented in the form of I-OR-AND

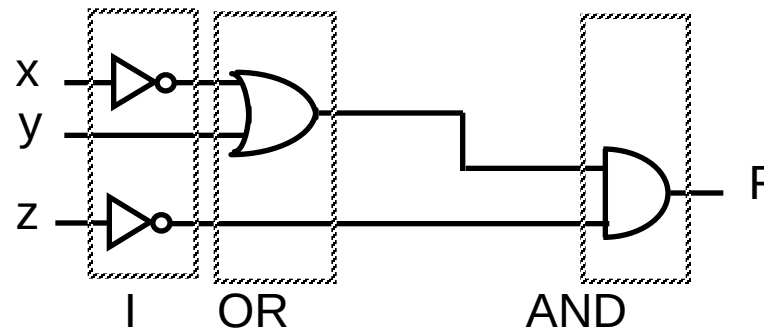
If we implement a Karnaugh map using 0-cells, the complement of F , i.e., F' , can be obtained. Thus, by complementing F' using DeMorgan's theorem F can be obtained

$$F(x,y,z) = (0,2,6)$$



$$F' = xy' + z$$

$$F = (xy')z' = (x' + y)z'$$



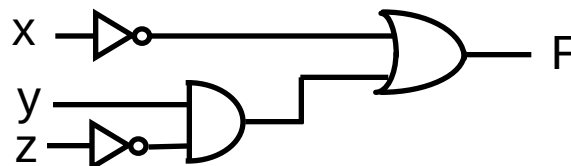
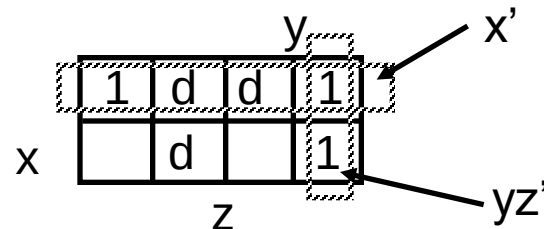
IMPLEMENTATION OF K-MAPS - Don't-Care Conditions -

In some logic circuits, the output responses for some input conditions are don't care whether they are 1 or 0.

In K-maps, don't-care conditions are represented by d's in the corresponding cells.

Don't-care conditions are useful in minimizing the logic functions using K-map.

- Can be considered either 1 or 0
- Thus increases the chances of merging cells into the larger cells
--> Reduce the number of variables in the product terms



COMBINATIONAL LOGIC CIRCUITS

Half Adder

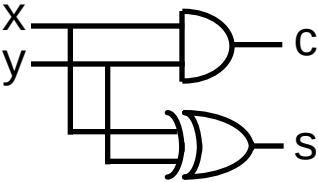
x	y	c	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

	y
x	0
	1

c = xy

	y
x	0
	1

s = xy' + x'y
= x ⊕ y



Full Adder

x	y	c _{n-1}	c _n	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

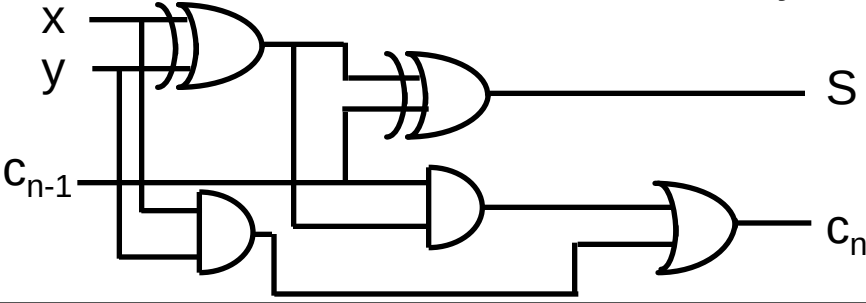
	y
x	0
	0
	1
	0

c_n

	y
x	0
	1
	0
	1

s

c_n = xy + xc_{n-1} + yc_{n-1}
= xy + (x ⊕ y)c_{n-1}
s = x'y'c_{n-1} + x'yc'_{n-1} + xy'c'_{n-1} + xyc_{n-1}
= x ⊕ y ⊕ c_{n-1} = (x ⊕ y) ⊕ c_{n-1}



COMBINATIONAL LOGIC CIRCUITS

Other Combinational Circuits

Multiplexer

Encoder

Decoder

Parity Checker

Parity Generator

etc

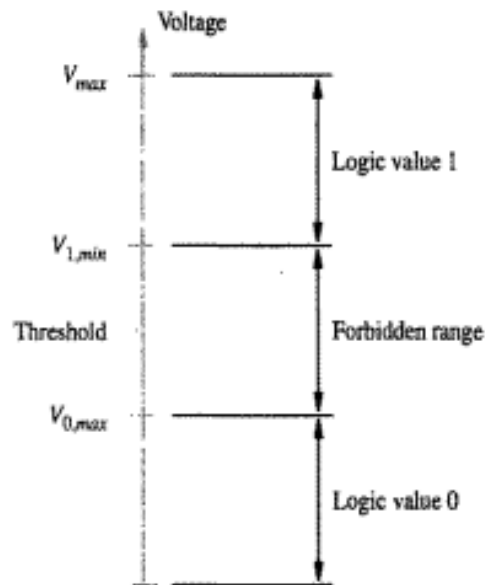
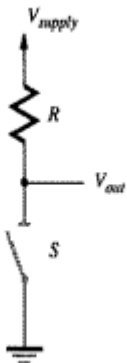


Figure A.10 Representation of logic values by voltage levels.

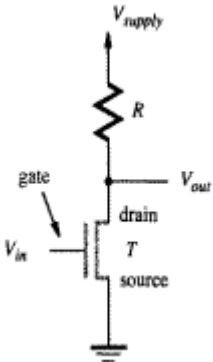
To establish a correspondence between voltage and logic values or states, a concept called **threshold** is used.

In practical situations, the voltage at any point in an electronic circuit undergoes small random variations for variety of reasons. It is called **noise**

Not Gate

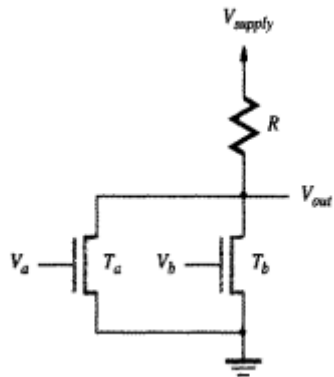
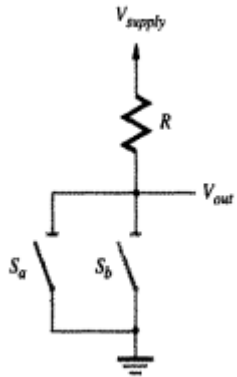


S - Close	V-out = 0
S - Open	V-out =1

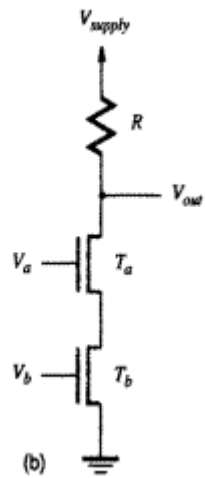


Vin	Switch(T)	V out	
0(0)	Open	Vsupply	1
Vsupply(1)	Close	0	0

NOR Gate



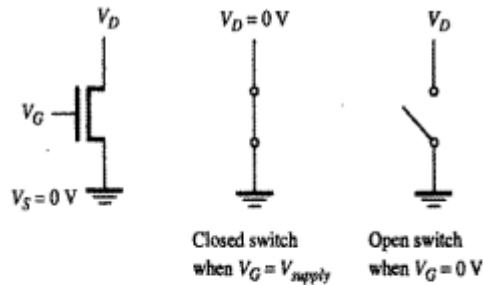
NAND Gate



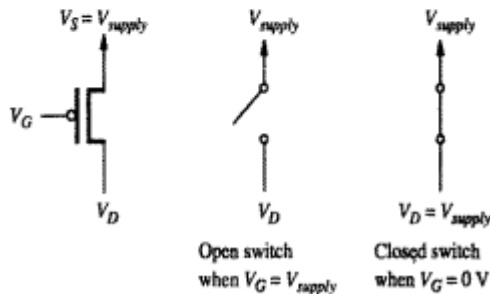
CMOS Circuits

Metal-Oxide Semiconductor (MOS) transistors

- Two types of CMOS transistors – NMOS and PMOS
- NMOS uses n-channel transistor and PMOS uses p-channel transistor
- The naming convention is due to the type of current flows through these transistors.



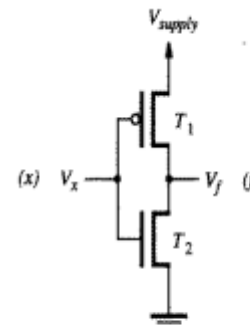
(a) NMOS transistor



(b) PMOS transistor

CMOS – Complementary MOS

Both T1 and T2 operate in complementary



(a) Circuit

x	V_x	T_1	T_2	V_f	f
0	low	on	off	high	1
1	high	off	on	low	0

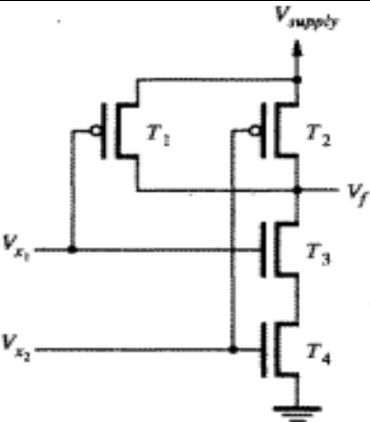
(b) Truth table and transistor states

Figure A.15 CMOS realization of a NOT gate.

CMOS Circuits

- The key feature of CMOS circuits is that transistors T1 and T2 operate in complementary fashion;
 - when one is on, the other is off. Hence there is always a closed path from the output point to either V_{supply} or ground.
 - There is no closed path between V_{supply} and ground at any point of time except during transition period when the transistors are changing their states.
 - This means that the circuit does not dissipate appreciable power when it is in a steady state.
 - It dissipates power only when it is switching from one logic to another.
 - Therefore, power dissipation in this circuit is dependent on the rate at which state changes take place.

CMOS NAND and NOR gates

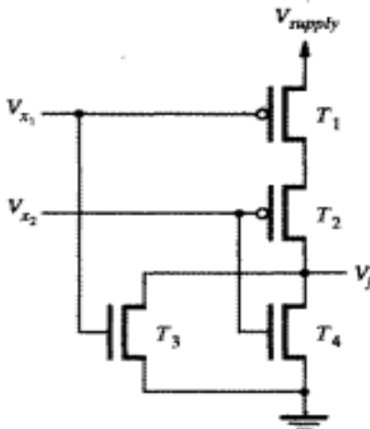


(a) Circuit

x_1	x_2	T_1	T_2	T_3	T_4	f
0	0	on	on	off	off	1
0	1	on	off	off	on	1
1	0	off	on	on	off	1
1	1	off	off	on	on	0

(b) Truth table and transistor states

Figure A.17 CMOS realization of a NAND gate.



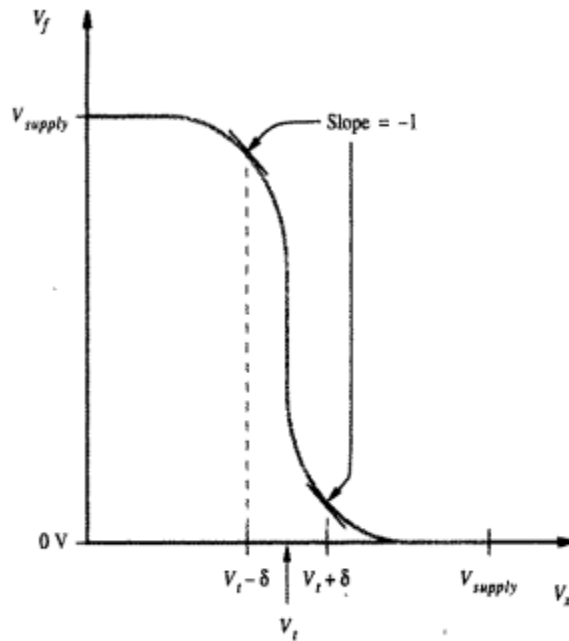
(a) Circuit

x_1	x_2	T_1	T_2	T_3	T_4	f
0	0	on	on	off	off	1
0	1	on	off	off	on	0
1	0	off	on	on	off	0
1	1	off	off	on	on	0

(b) Truth table and transistor states

Figure A.18 CMOS realization of a NOR gate.

Voltage transfer characteristic for the CMOS inverter



V_t is a threshold voltage

Figure A.20 The voltage transfer characteristic for the CMOS inverter.

Propagation delay and transition time

- Logic circuits do not switch instantaneously from one state to another
- When a state change takes place at the input, a delay is encountered before the corresponding change at the output. This is called a propagation delay..
- Another important parameter is transition time, which is normally measured between the 10% and 90% points of the signal swing.

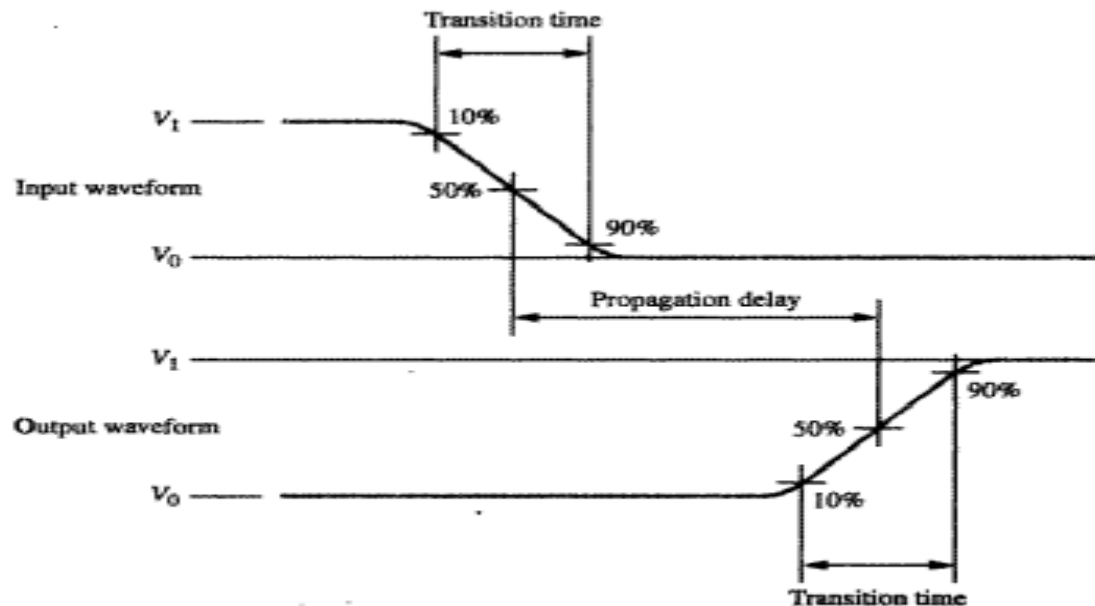


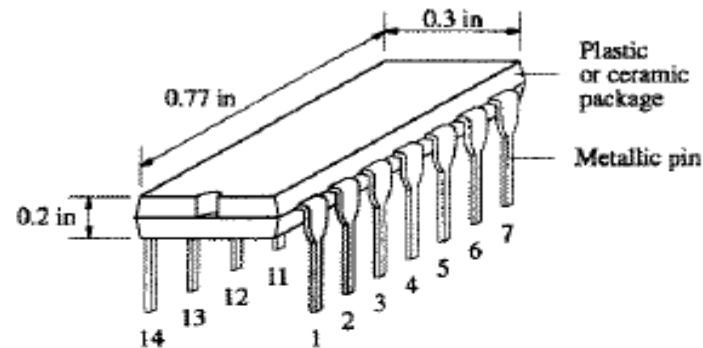
Figure A.21 Definition of propagation delay and transition time.

Fan-in and Fan-out constraints

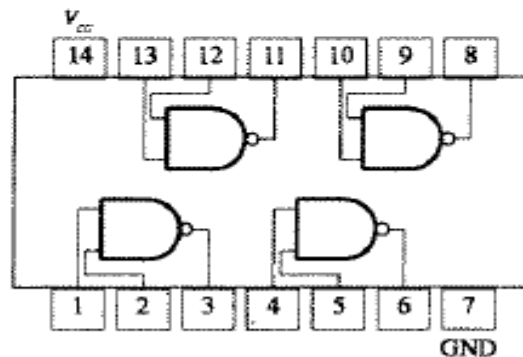
- Fan-in is the number of inputs that a logic gate can handle.
 - For example, if a logic gate can take 2 inputs, its fan-in is 2. Larger fan_in means that the gate can accept more signals as inputs.
 - Fan-in determines how complex the gate can be in terms of processing multiple inputs.
- Fan-out: This is the number of outputs a logic gate can drive or feed into. In other words, it refers to how many subsequent gates can be connected to the output of a given gate.
 - A high fan-out means that the output of the gate can be connected to more gates without signal degradation.
 - Fan-out is critical because exceeding a gate's fan-out capacity can result in signal integrity issues, such as slower signal propagation or weakened signals.
- Each transistor in a CMOS gate contributes a certain amount of capacitance. As the capacitance increases, the circuit becomes slower and its signal levels and noise levels become worse.
- Cascading is the solution for larger fan-in and using more gates is a solution for larger fan-out.

IC Circuits

Small scale integrated circuits



(a) Physical appearance



(b) Schematic of an integrated circuit providing four 2-input NAND gates

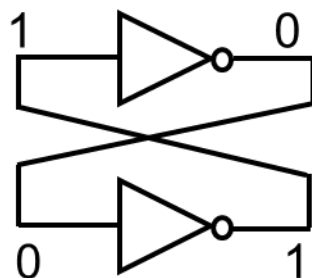
Figure A.23 A 14-pin integrated circuit package.

Flip-flops

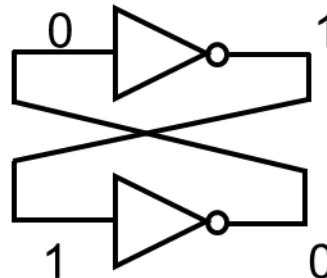
- Flip-flops are fundamental building blocks in digital electronic circuits, primarily used for storing and transferring binary data (1s and 0s). They are a type of bistable multivibrator, which means they have two stable states: they can store either a "0" or a "1".
- The circuit can "flip" from one state (0) to another (1) and "flop" back to the original state. This alternating between two stable states when triggered by a clock or input signal gives rise to the term "flip-flop."

Characteristics

- 2 stable states
- Memory capability
- Operation is specified by a Characteristic Table



0-State

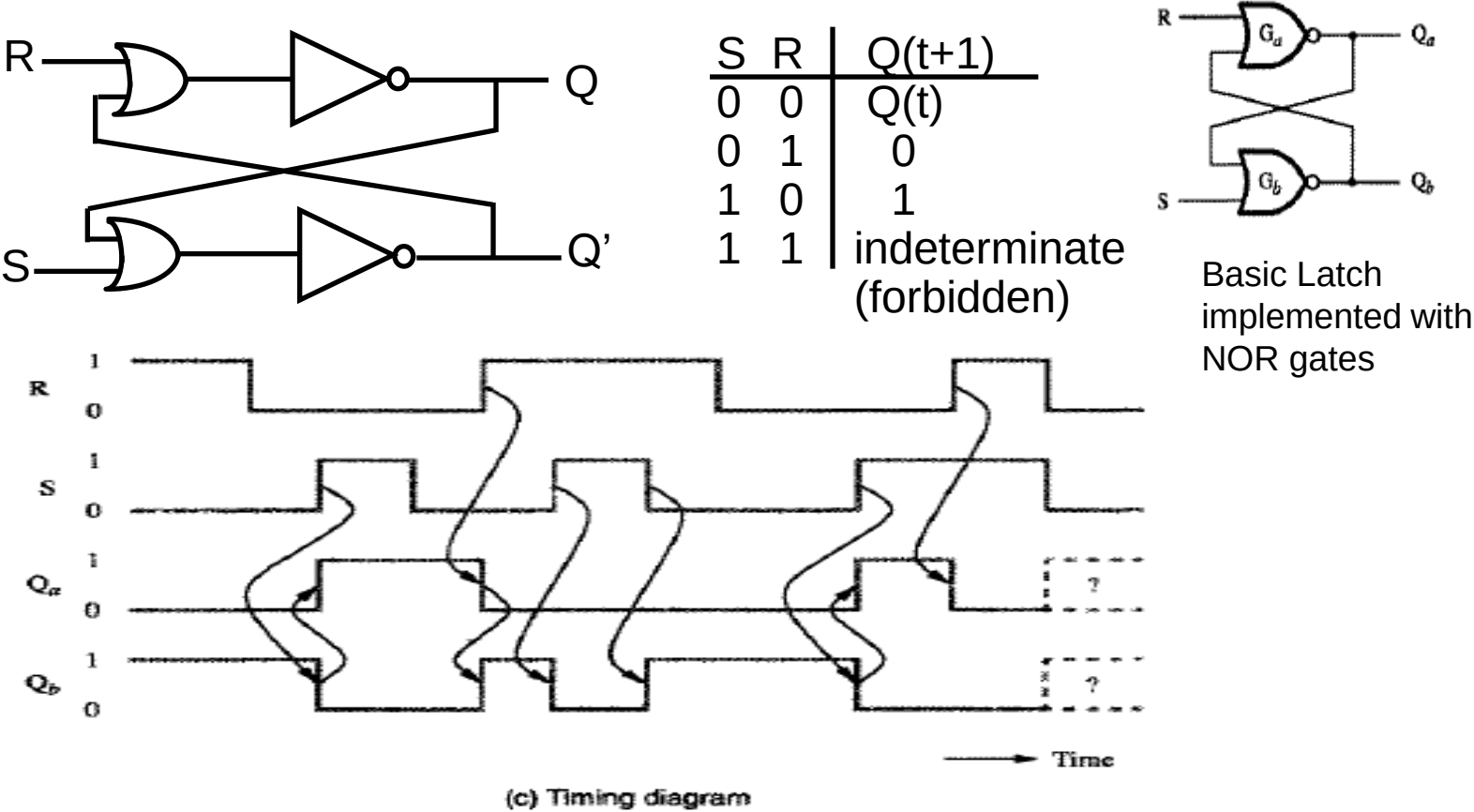


1-State

SR FLIP- FLOP

In order to be used in the computer circuits, state of the flip flop should have input terminals and output terminals so that it can be set to a certain state, and its state can be read externally.

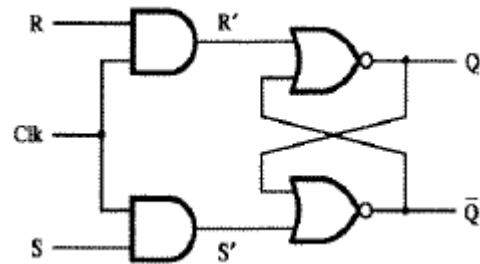
If S is set to 1 with R equal to 0, Q and Q' will become 1 and 0 respectively, and remain in this state after S is returned to 0. Hence, this circuit constitutes a memory element. Or a latch.
Q is considered as output and Q' is not used in practice. S and R are called set and reset



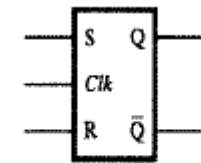
Gated SR FLIP- FLOP

Many applications require that the time at which a latch is set or reset be controlled from an input other than R and S. This input is termed a clock input. The circuit is called a gated latch.

- When the clk is equal to 1, points S' and R' follow the inputs S and R respectively.
- On the other hand, when clk = 0, the S' and R' points are equal to 0 and no change in the state of the latch can take place.



Clk	S	R	Q(t+1)
0	x	x	Q(t) (no change)
1	0	0	Q(t) (no change)
1	0	1	0
1	1	0	1
1	1	1	x



(d) Graphical symbol

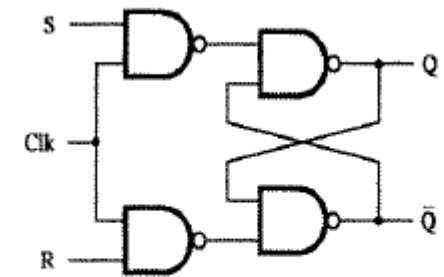
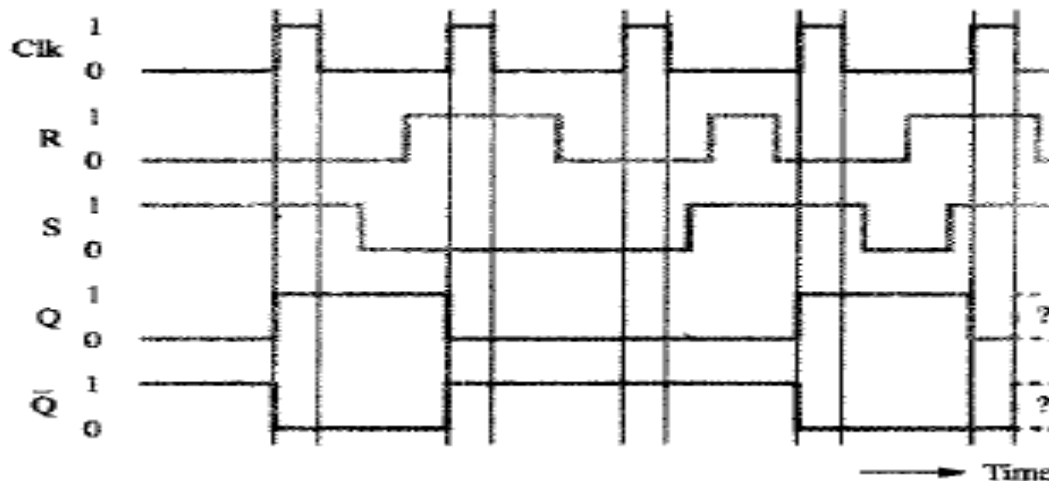
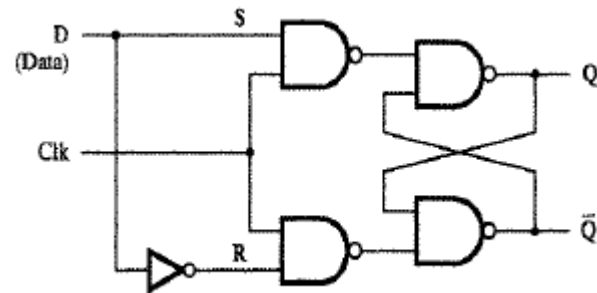


Figure A.26 Gated SR latch implemented with NAND gates.

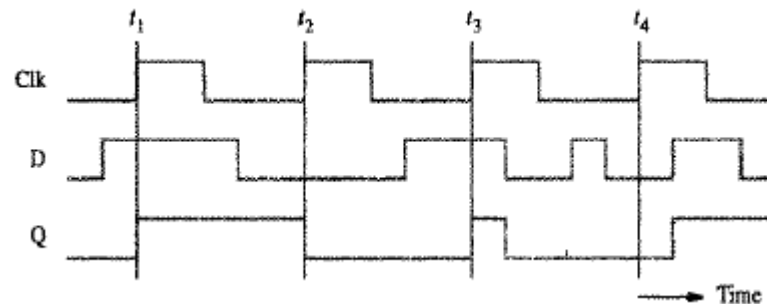
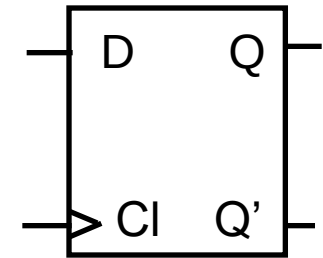
D-LATCH

D-Latch

Forbidden input values are forced not to occur by using an inverter between the inputs



D	Q(t+1)
0	0
1	1



- D flip-flop samples the D input at the time the clock is high and stores the information until subsequent clock pulse arrives

RS-LATCH WITH PRESET AND CLEAR INPUTS

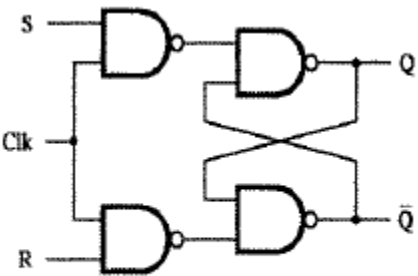
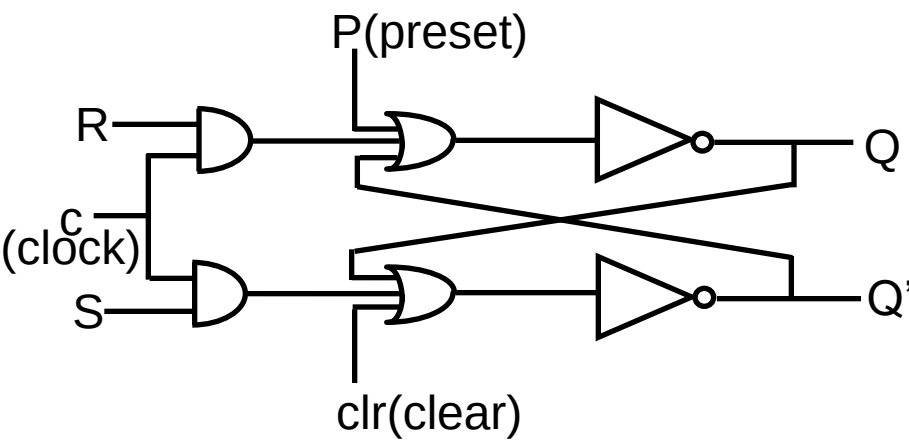
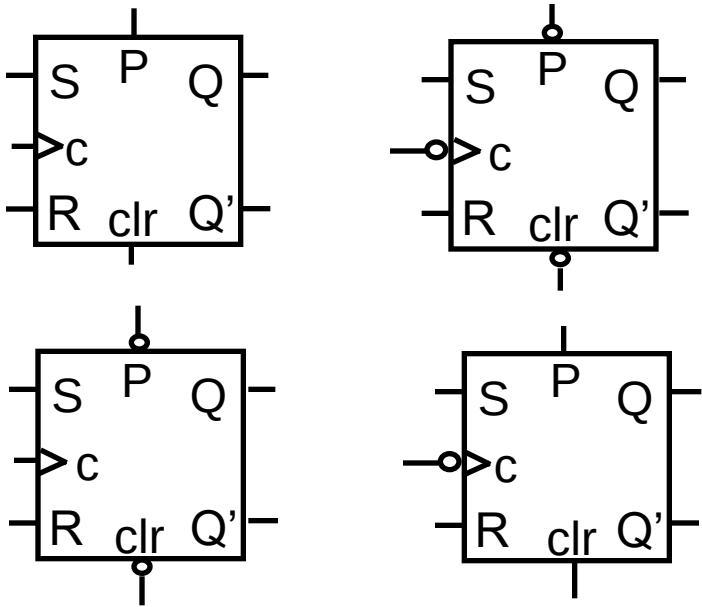


Figure A.26 Gated SR latch implemented with NAND gates.

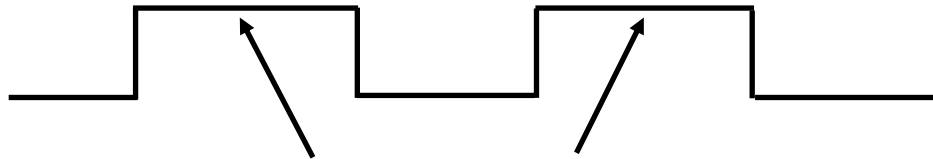


EDGE-TRIGGERED FLIP FLOPS

Characteristics

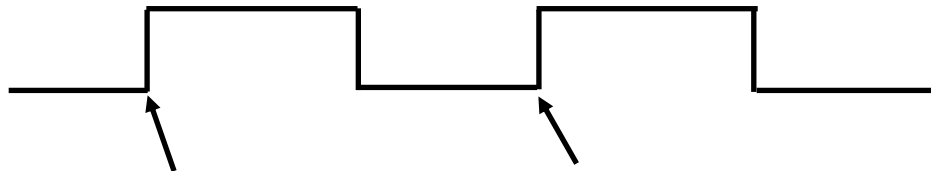
- State transition occurs at the rising edge or falling edge of the clock pulse

Latches



respond to the input only during these periods

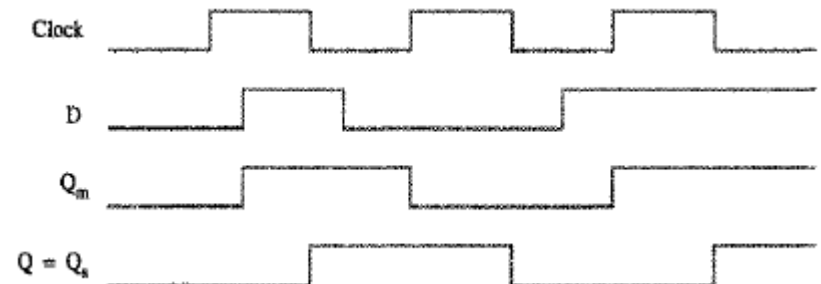
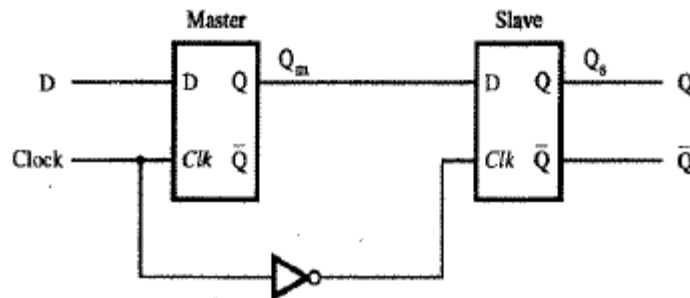
Edge-triggered Flip Flops (positive)



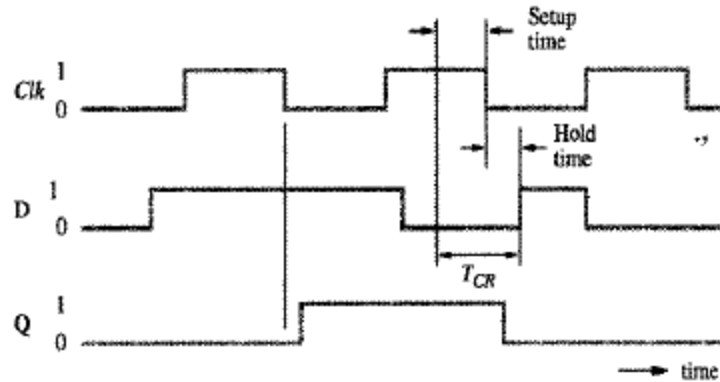
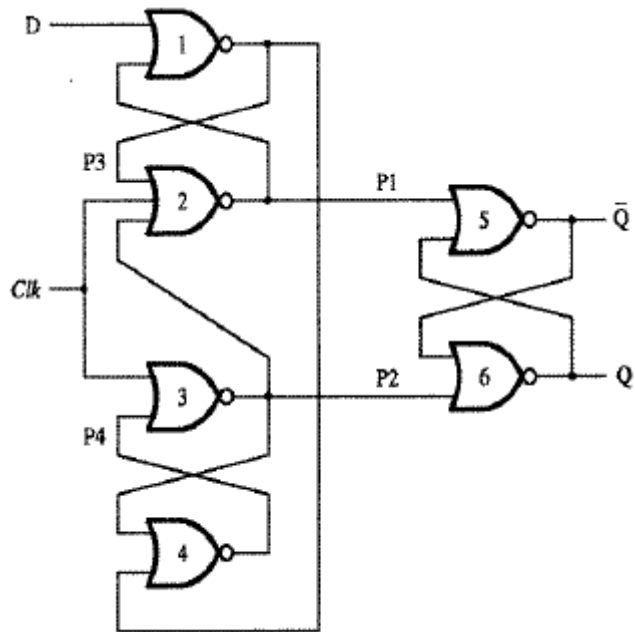
respond to the input only at this time

Master Slave Flip-flops

- In previously studied flip-flops, we assume that while $\text{clk}=1$, S and R do not change. However, the outputs will respond immediately to any change in S or R input during time.
- Similarly, for D latch $Q=D$ while $\text{clk} =1$.
- In such circuits, immediate propagation of logic conditions from inputs (S,R and D) to the latch outputs may lead to incorrect operations.
- This is undesirable for counters and shift-registers.
- Two gated latches can be connected to form the master-slave organization eliminates this problem.
- The first, the master is connected to the inputs and its output is connected to slave. When Clock =1, a 1to0 transition of the clock isolates the master from the input and transfers the contents of the master stage to the slave stage.
- The function of the slave is to hold the value at the output of the flipflop while master stage is being set up to the next state value determined by the input.
- The new state is transferred from master to the slave after the 1 to 0 transition on Clock.
- At this stage the master is isolated from the inputs so that further changes in the latch input will not affect this transfer.

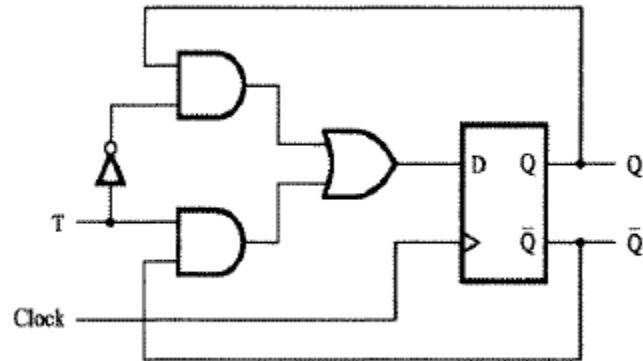


Negative edge triggered D flip flop

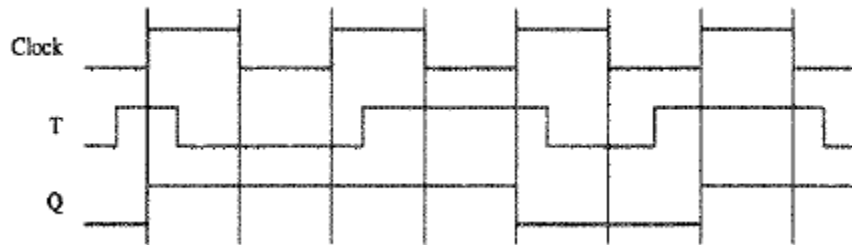
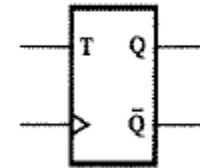


T Flip-flop

Changes its state every clock cycle.

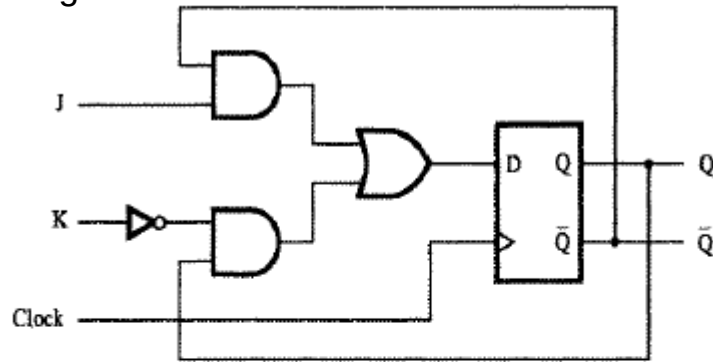


T	$Q(t+1)$
0	$Q(t)$
1	$\bar{Q}(t)$



JK Flip-flop

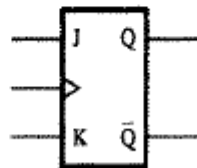
- JK flip-flop combines the behavior of SR and T flip-flops
- The first 3 entries in the truth table defines the same behavior as SR flip-flop
- When $J=K=1$, the next state is defined as the complement of the present state of the flip-flop. i.e. when $J=K=1$ the flip-flop toggles, reversing its present state.
- JK flip-flop is versatile, it can be used to store data like RS flip-flop. It can also be used to built counters and shift registers, because it behaves like T flip-flop when its inputs are connected together.



(a) Circuit

J	K	$Q(t+1)$
0	0	$Q(t)$
0	1	0
1	0	1
1	1	$\bar{Q}(t)$

(b) Truth table

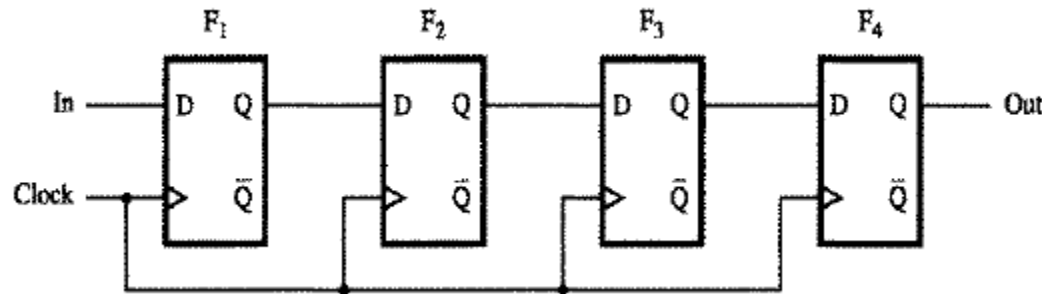


(c) Graphical symbol

- JK flip-flop can be implemented using two D flip-flops connected as $D = JQ' + J'Q = JQ' \text{ AND } J'Q$

Registers and Shift Registers

- Individual flip-flops can be used to store one bit.
- In machines in which data stored are handled in words consisting of many bits (2,4,8,16,32,64 etc.
- It is convenient to arrange a number of flip-flops into a common structure called register.
- The data are written (loaded) into or read from all flip-flops of a register at a same time, hence, the operation of all flip-flops in a register is synchronized by a common clock.
- Processing of digital data often requires the capability to (1) shift and (2) rotate the data.
 - A simple mechanism for realizing both these operations is a register whose contents may be shifted to the right or left one bit position at a time
 - It is called a shift register

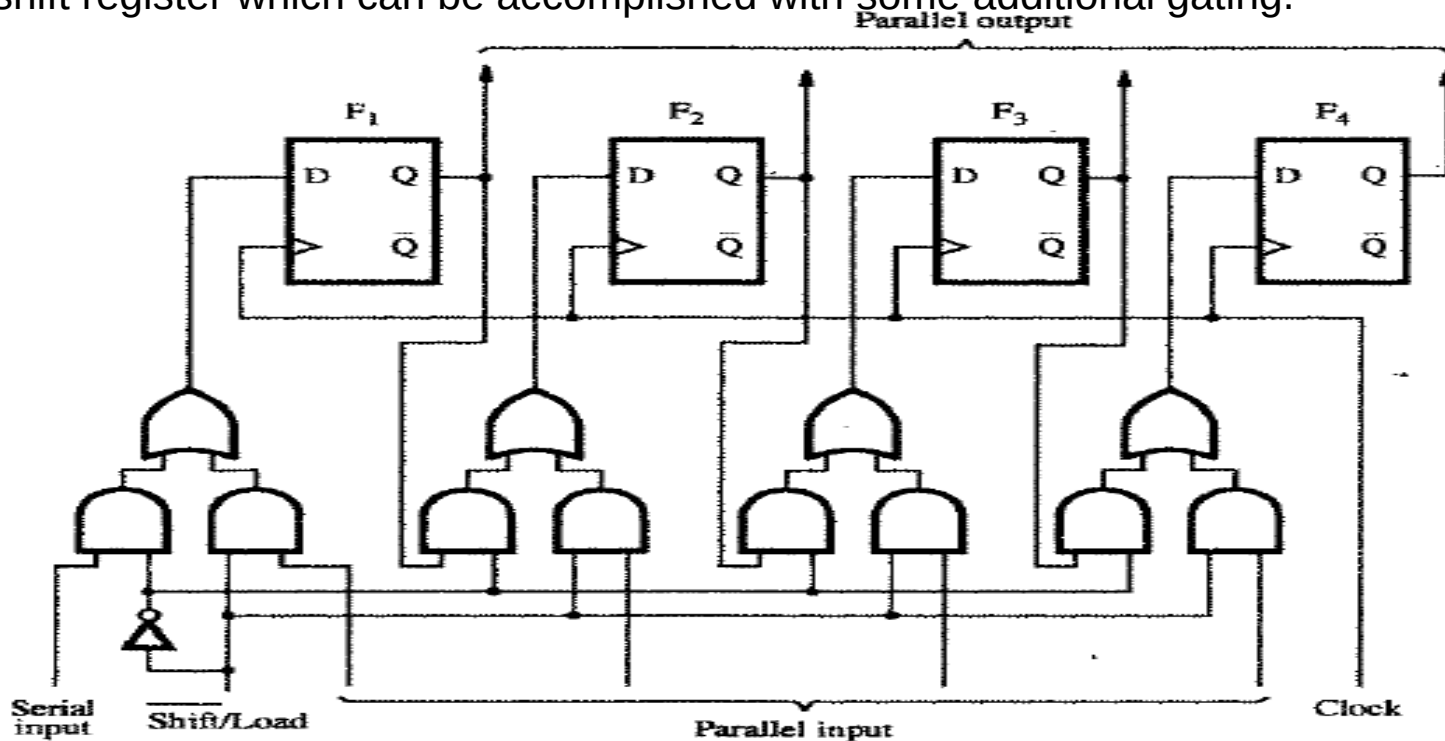


A simple shift register

- It consists of D flip-flops connected so that each clock pulse will cause the transfer of the state of F_i to F_{i+1} , effecting a “right Shift”.
- Data is shifted serially into and out of the register
- The rotation of the data can be implemented by connecting Out to In

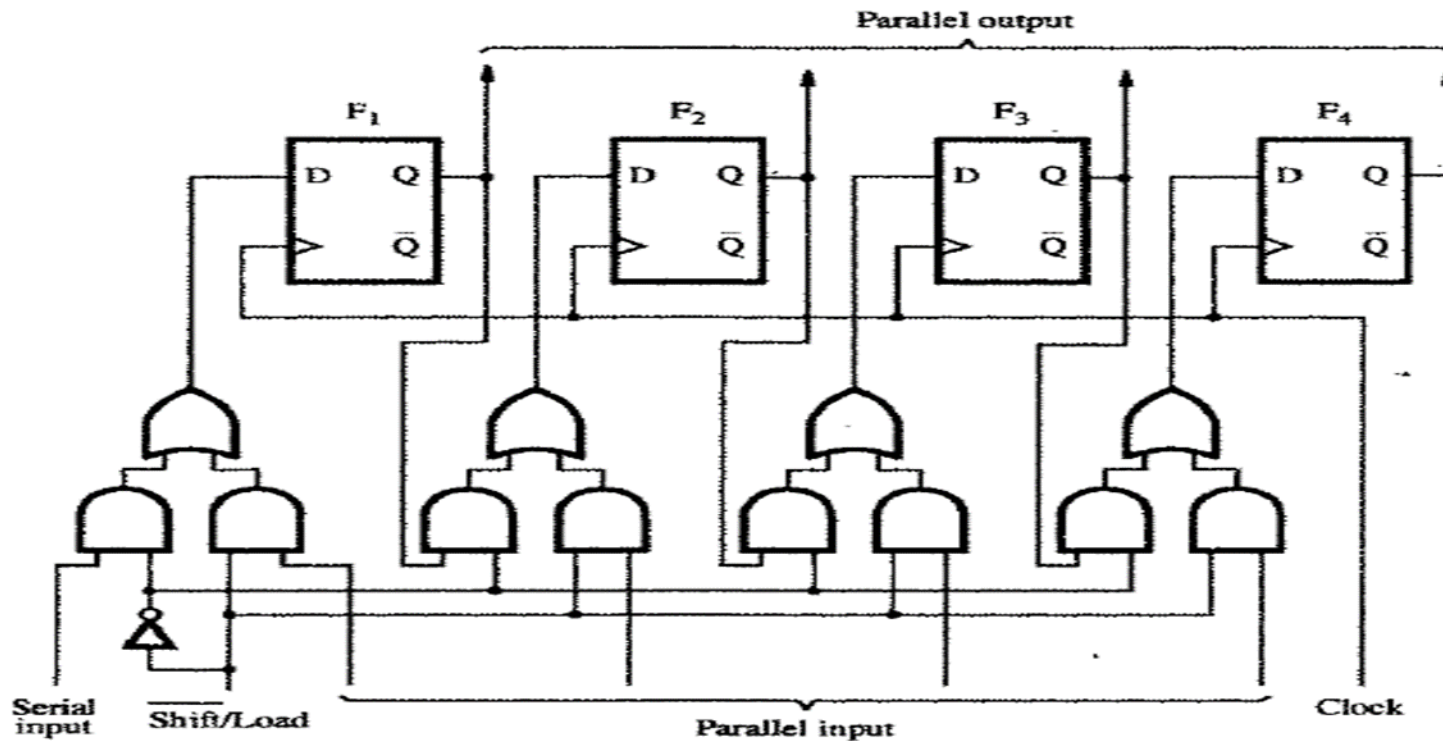
Parallel/Serial Shift register

- Shift register operations require that its contents be shifted exactly one position for each clock pulse.
- D flip-flops and gated flip-flops, the D input quickly propagates to the output. It will continue to propagate across all flip-flops.
- There is no control over the number of shifts that will take place during a single clock pulse. The number of shifts depends on the propagation delays and duration of the clock pulse
- Hence the solution is use either master-slave or edge triggered flip-flops
- One useful form of shift register is one that can be loaded and read parallel, called parallel shift register which can be accomplished with some additional gating.



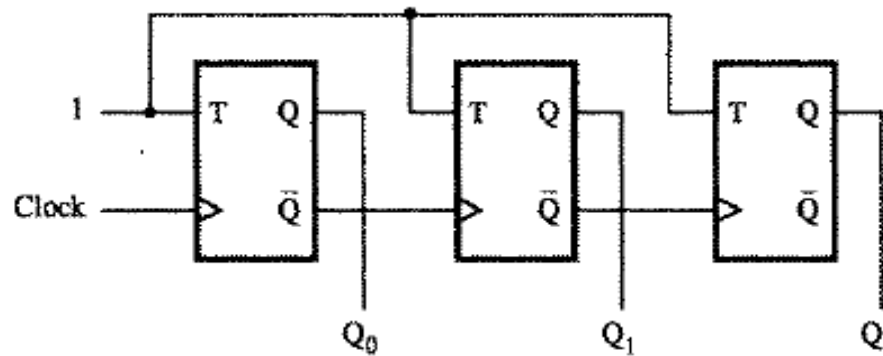
Parallel/Serial Shift register

- Here is a 4 bit register constructed with D flip-flops.
- It can be loaded either serially or in parallel
- When register is clocked, a shift takes place if shift'/Load=0:
- Otherwise, a parallel load is performed.

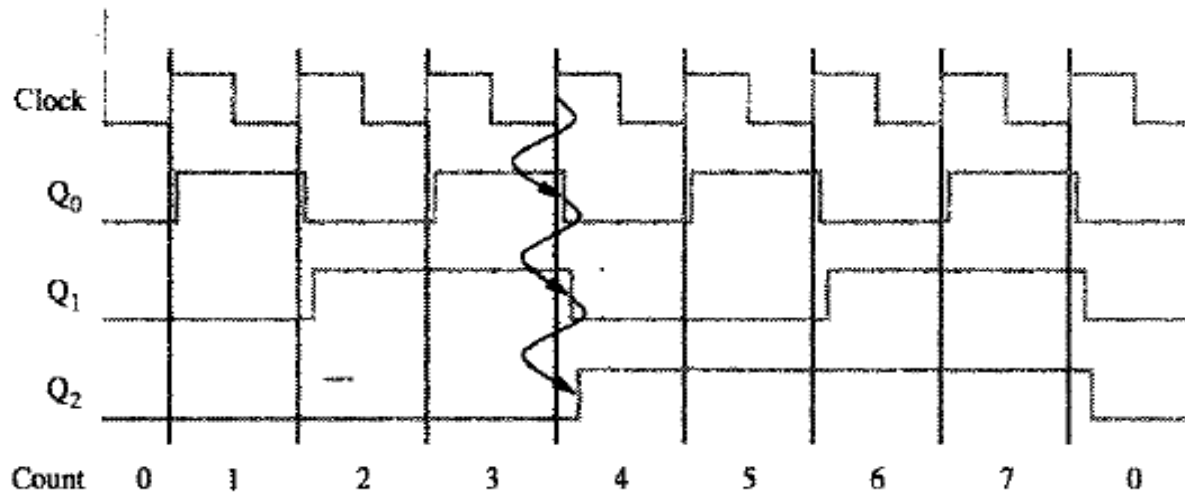


Counters

- Another application of flip-flops is implementation of counter circuits
- Counters are also used to generate control and timing signals.
- A 3 bit counter with T flip-flops is shown here



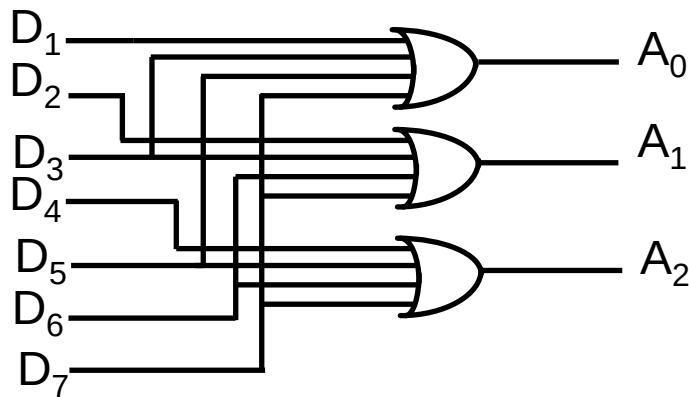
(a) Circuit



- When T input is 1, the flip-flop toggles, i.e. for each successive pulse Q_0 to change from 1 state to 0 state and back to 1 state.
- i.e. the output waveform of Q_0 has half frequency of the clock.
- Q_1 has the half frequency of Q_0 , Similarly, Q_2 has half frequency of Q_1
- Hence it is also called a ripple counter, because the effect of an input clock pulse ripples through the counter.

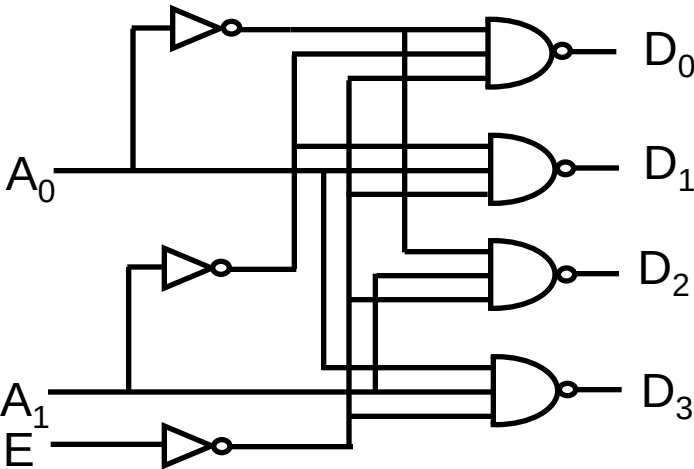
ENCODER/DECODER

Octal-to-Binary Encoder



2-to-4 Decoder

E	A ₁	A ₀	D ₀	D ₁	D ₂	D ₃
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0
1	d	d	1	1	1	1



Decoders

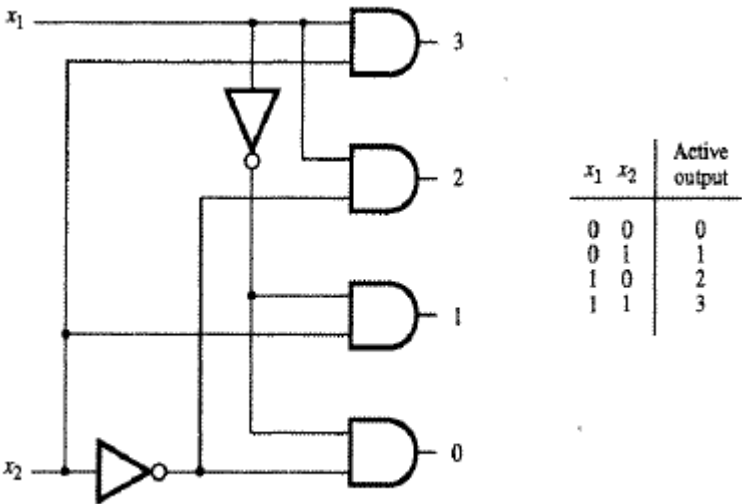


Figure A.36 A two-input to four-output decoder.

A decoder driving 7 segment display

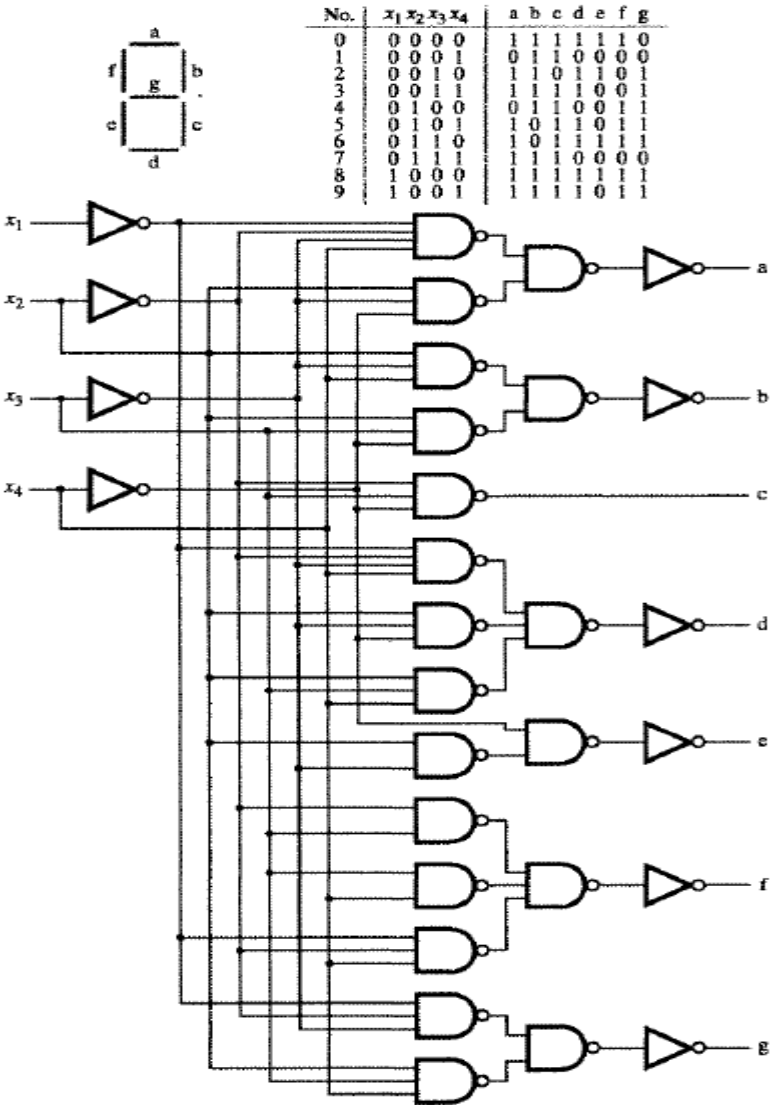
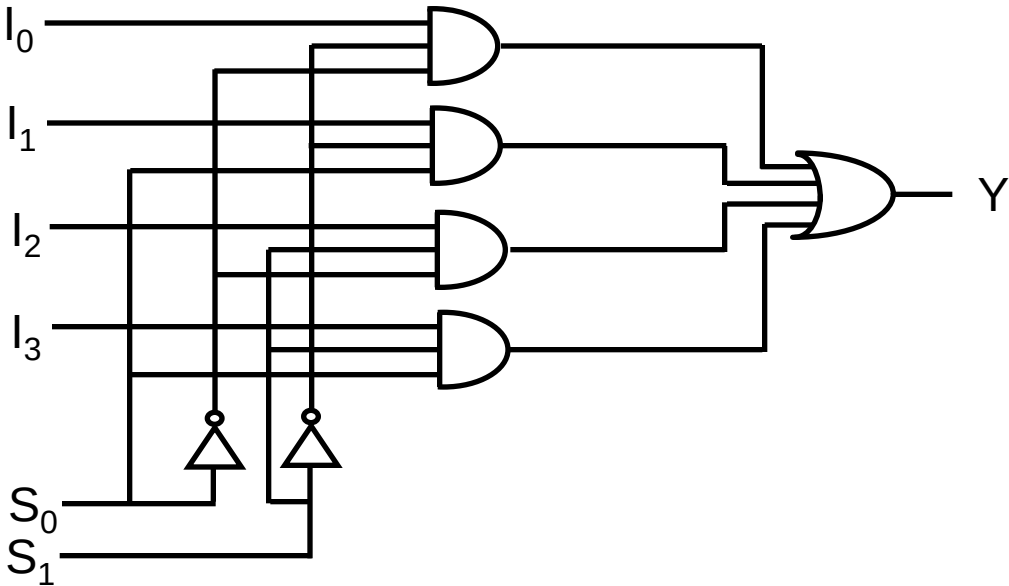


Figure A.37 A BCD to seven-segment display decoder.

MULTIPLEXER

4-to-1 Multiplexer

Select		Output
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



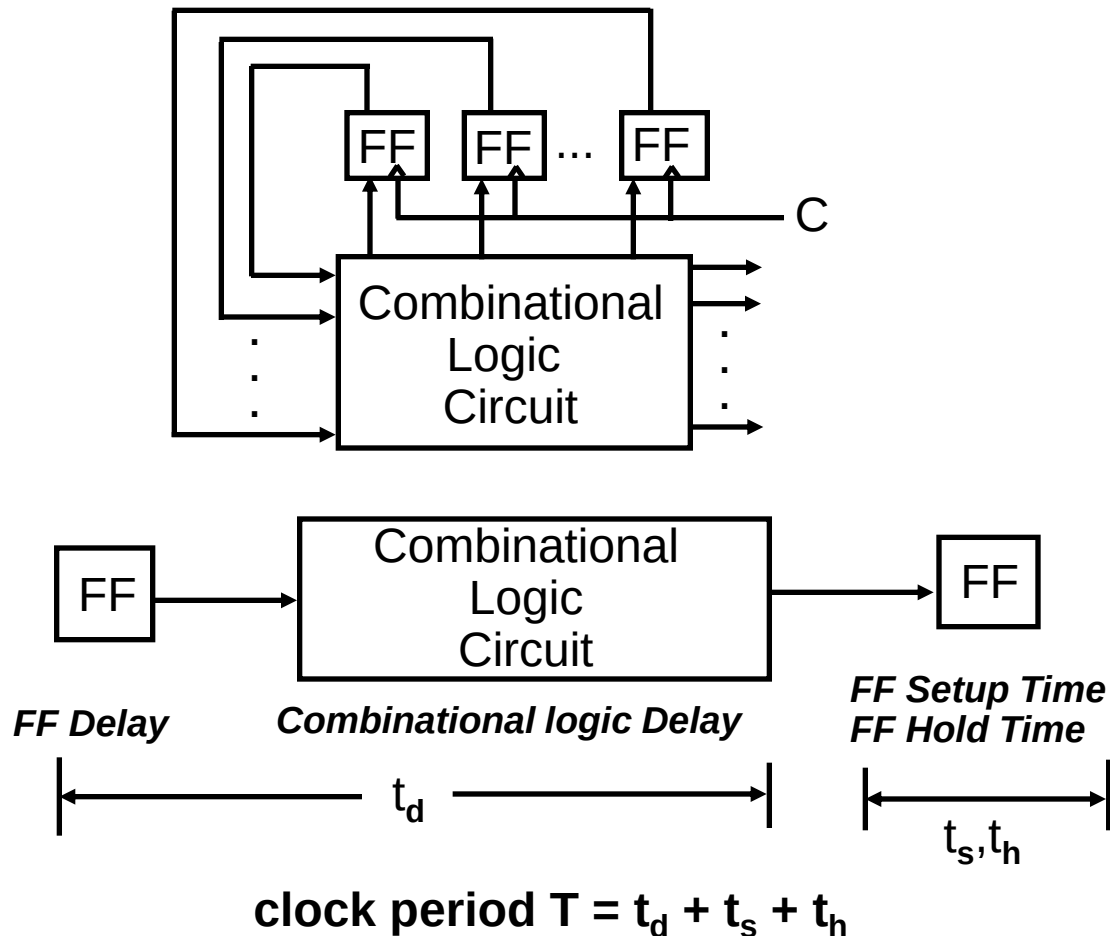
Sequential Circuits

- A combinational circuit is the one whose output is determined entirely by its present inputs.
- Examples: Multiplexers and decoders
- Sequential circuits output depends on both the present inputs and on the sequence of previous inputs.

CLOCK PERIOD

**Clock period determines how fast the digital circuit operates.
How can we determine the clock period ?**

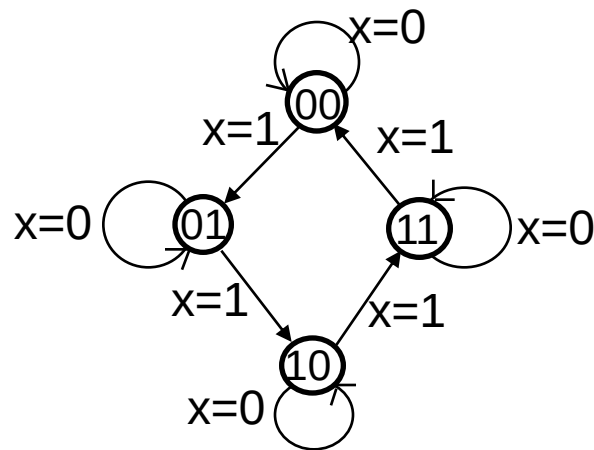
Usually, digital circuits are sequential circuits which has some flip flops



DESIGN EXAMPLE

Design Procedure:
Specification $\Rightarrow$ State Diagram $\Rightarrow$ State Table $\Rightarrow$
Excitation Table $\Rightarrow$ Karnaugh Map $\Rightarrow$ Circuit Diagram

Example: 2-bit Counter $\rightarrow$ 2 FF's



current state		input	next state		FF inputs			
A	B		A	B	Ja	Ka	Jb	Kb
0	0	0	0	0	0	d	0	d
0	0	1	0	1	0	d	1	d
0	1	0	0	1	0	d	d	0
0	1	1	1	0	1	d	d	1
1	0	0	1	0	d	0	0	d
1	0	1	1	1	d	0	1	d
1	1	0	1	1	d	0	d	0
1	1	1	0	0	d	1	d	1

	B
	1
d	d
d	d

A

x

Ja

Ja = Bx

	B
d	d
d	d
	1

A

x

Ka

Ka = Bx

	B
	d
1	d
1	d
	d

A

x

Jb

Jb = x

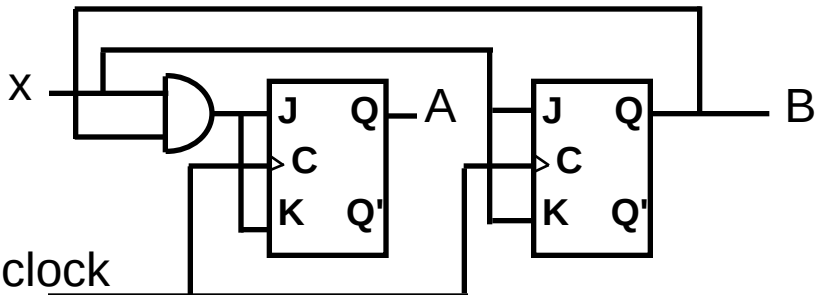
	B
d	
d	1
d	1
d	

A

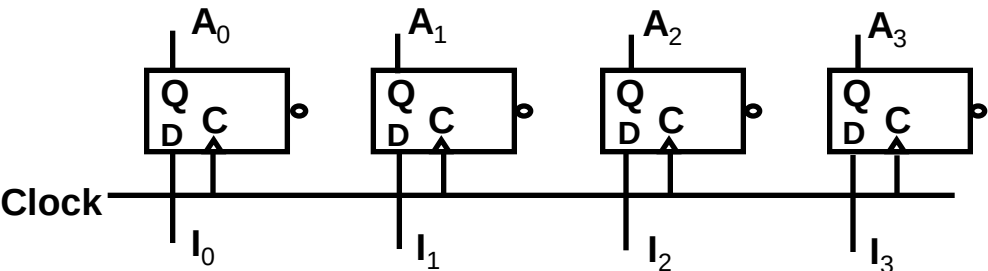
x

Kb

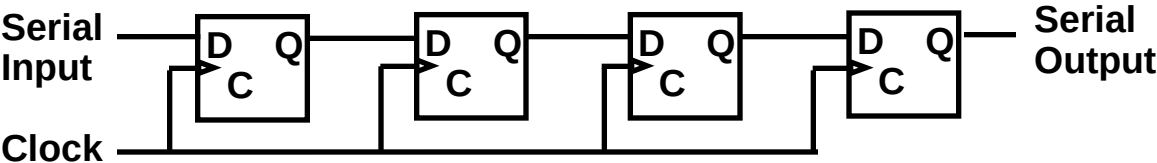
Kb = x



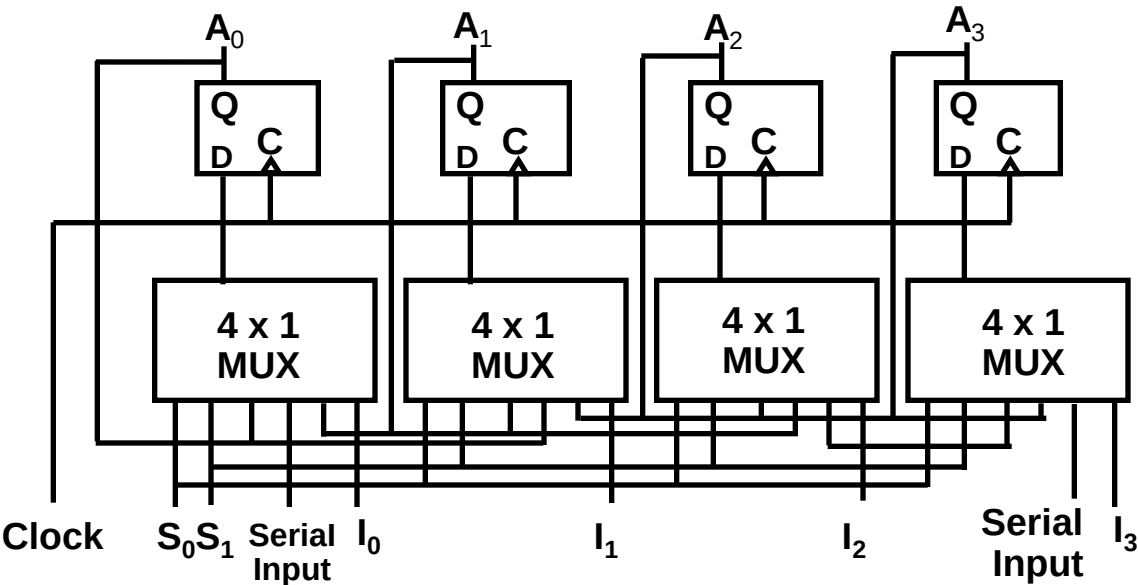
SEQUENTIAL CIRCUITS - Registers



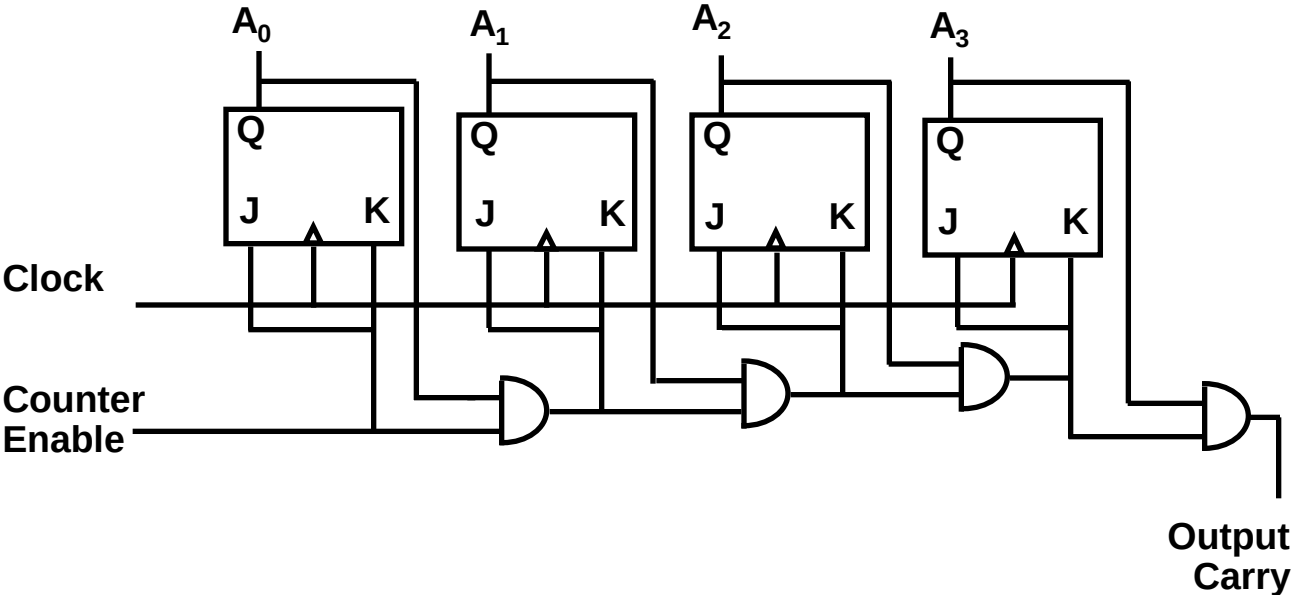
Shift Registers



Bidirectional Shift Register with Parallel Load



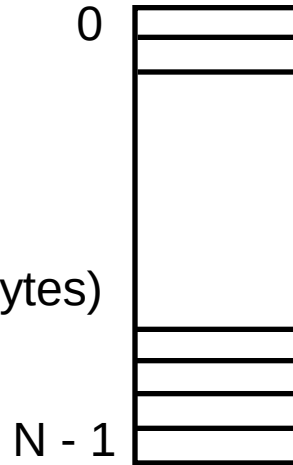
SEQUENTIAL CIRCUITS - Counters



MEMORY COMPONENTS

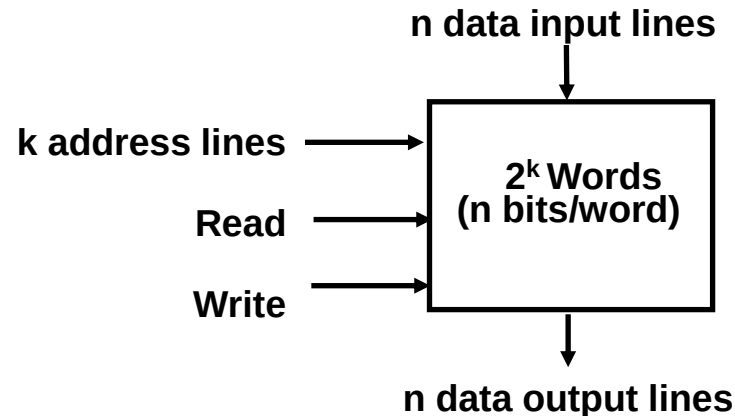
Logical Organization

words
(byte, or n bytes)



Random Access Memory

- Each word has a unique address
- Access to a word requires the same time independent of the location of the word
- Organization

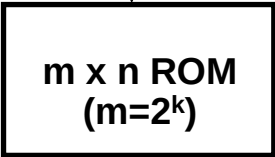


READ ONLY MEMORY(ROM)

Characteristics

- Perform read operation only, write operation is not possible
- Information stored in a ROM is made permanent during production, and cannot be changed
- Organization

k address input lines



n data output lines

Information on the data output line depends only on the information on the address input lines.

--> Combinational Logic Circuit

$$\begin{aligned} X_0 &= A'B' + B'C \\ X_1 &= A'B'C + A'BC' \\ X_2 &= BC + AB'C' \\ X_3 &= A'BC' + AB' \\ X_4 &= AB \end{aligned}$$



$$\begin{aligned} X_0 &= A'B'C' + A'B'C + AB'C \\ X_1 &= A'B'C + A'BC' \\ X_2 &= A'BC + AB'C' + ABC \\ X_3 &= A'BC' + AB'C' + AB'C \\ X_4 &= ABC' + ABC \end{aligned}$$

Canonical minterms

address	Output				
	ABC	X ₀	X ₁	X ₂	X ₃
000	1	0	0	0	0
001	1	1	0	0	0
010	0	1	0	1	0
011	0	0	1	0	0
100	0	0	1	1	0
101	1	0	0	1	0
110	0	0	0	0	1
111	0	0	1	0	1

TYPES OF ROM

ROM

- Store information (function) during production
- Mask is used in the production process
- Unalterable
- Low cost for large quantity production --> used in the final products

PROM (Programmable ROM)

- Store info electrically using PROM programmer at the user's site
- Unalterable
- Higher cost than ROM -> used in the system development phase
-> Can be used in small quantity system

EPROM (Erasable PROM)

- Store info electrically using PROM programmer at the user's site
- Stored info is erasable (alterable) using UV light (electrically in some devices) and rewriteable
- Higher cost than PROM but reusable --> used in the system development phase. Not used in the system production due to eras ability

INTEGRATED CIRCUITS

Classification by the Circuit Density

- SSI -** several (less than 10) independent gates
- MSI -** 10 to 200 gates; Perform elementary digital functions;
Decoder, adder, register, parity checker, etc
- LSI -** 200 to few thousand gates; Digital subsystem
Processor, memory, etc
- VLSI -** Thousands of gates; Digital system
Microprocessor, memory module

Classification by Technology

- TTL -** Transistor-Transistor Logic
Bipolar transistors
NAND
- ECL -** Emitter-coupled Logic
Bipolar transistor
NOR
- MOS -** Metal-Oxide Semiconductor
Unipolar transistor
High density
- CMOS -** Complementary MOS
Low power consumption